



ClinicalPrep AI — Technical Documentation

System Architecture, Tech Stack, Modular Monolith Specifications & Operational Tooling (v1.0 – v4.0+)

1. Architectural Vision: The Modular Monolith

ClinicalPrep AI v2.0 transitions from a single-file Streamlit application into a **Decoupled Modular Monolith**. The system decouples the client-side user interface (React Vite on Vercel) from backend business logic (FastAPI on Render), maintaining strict internal service boundary segregation inside Python.

```
ACE_Chatbot/
├── frontend/                                <-- React (Vite) Application
│   ├── src/
│   │   ├── components/                    <-- ChatBubbles, Header, VoiceRecorder, TokenCounter
│   │   ├── hooks/                        <-- useAudioRecorder, useChatStream
│   │   ├── App.jsx
│   │   └── package.json
│   └── package.json
├── backend/                                <-- Unified FastAPI Modular Monolith
│   ├── app/
│   │   ├── main.py                       <-- Unified FastAPI Routing Entrypoint
│   │   ├── core/                         <-- Security, CORS, Config, Slowapi Rate Limiter
│   │   └── services/                     <-- Isolated Business Services
│   │       ├── intake/                   <-- Pydantic v2 Slot-Filling & State Machine
│   │       ├── audio/                    <-- STT (Whisper API) & TTS (tts-1) Engine
│   │       ├── guardrails/               <-- 3-Layer Red-Flag Triage & Intent Router
│   │       ├── pdf/                      <-- ReportLab / PyPDF Generator
│   └── requirements.txt
```

2. Comprehensive Technology Stack & Production Tooling

Component	Technology	Purpose & Production Rationale
Frontend Framework	React (Vite) + JavaScript	Sub-second build times, ultra-lightweight client bundle, native mobile drawer animation support.
Frontend Styling	Tailwind CSS + Framer Motion	Utility-first CSS custom palette ("Chocolate Truffle") and smooth mobile touch gestures.
Backend Engine	FastAPI (Python 3.11+)	High-performance `asyncio` event loop, native Pydantic v2 data validation, OpenAPI docs.
LLM & Structured Output	OpenAI `gpt-4o-mini`	`chat.completions.parse` with `strict: true` for 100% schema-compliant JSON slot extraction.
Audio STT & TTS	Whisper API + `tts-1`	Accent-resilient speech-to-text transcription and low-latency voice synthesis streaming.
Observability & Tracing	Langfuse SDK	Prompt versioning, LLM call latency tracing, token cost accounting per patient turn.
Crash Reporting	Sentry (Python & React)	Real-time error logging, stack trace capture, and mobile audio permission failure tracking.
API Security & Rate Limit	Slowapi / Upstash Redis	IP-based rate limiting (10 voice requests/min) to prevent API quota drain and DOS.
Database & Vector Memory	Supabase PostgreSQL + pgvector	Relational user data storage and sub-100ms vector semantic search for RAG patient history.

3. Voice Processing Pipeline (Web Audio → Whisper → TTS)

The multi-modal audio engine operates asynchronously across 4 sequential stages:

- 1. Client Recording:** Browser `MediaRecorder` captures webm/wav audio blobs on user mic release.
- 2. STT Endpoint (`POST /api/v2/audio/transcribe`):** FastAPI receives multipart audio form data and forwards it to OpenAI Whisper, returning structured transcript text.
- 3. Chat Processing (`POST /api/v2/chat/step`):** Transcribed text passes through the 3-Layer Guardrail and Pydantic Slot Filler.
- 4. Voice Synthesis (`POST /api/v2/audio/speak`):** Text response streams through `tts-1` returning audio bytes played via browser HTML5 Audio.

4. Data Schemas & Pydantic Extraction Model

```
# app/services/intake/schemas.py
from pydantic import BaseModel, Field
from typing import Optional, List

class PatientSlotState(BaseModel):
    # Demographics
    patient_name: Optional[str] = Field(None, description="Patient full name")
    patient_age: Optional[int] = Field(None, description="Age in years")
    gender_identity: Optional[str] = Field(None, description="Gender identity")

    # History of Present Illness (HPI)
    chief_complaint: Optional[str] = Field(None, description="Primary symptom or reason for visit")
    onset_duration: Optional[str] = Field(None, description="When symptoms started")
    pain_severity: Optional[int] = Field(None, description="Severity score 1-10")
    symptom_pattern: Optional[str] = Field(None, description="Constant, intermittent, waves")
    current_medications: Optional[str] = Field(None, description="OTC or prescription meds taken")
    patient_goals: Optional[str] = Field(None, description="Specific questions for doctor")

    # Engine Control
    missing_fields: List[str] = Field(..., description="Unfilled mandatory slots")
    next_question: str = Field(..., description="1 empathetic question targeting highest priority missing slot")
```

5. Security, Secrets & Legal Compliance Engine

- Secrets Management:** Environment variables stored in local `.env` (git-ignored) and injected via Vercel/Render production dashboards.
- Data Privacy (India DPDP Act 2023):** Explicit consent screen at session start; transient session storage with automatic deletion after PDF export.
- HIPAA Readiness (US Target):** Azure OpenAI Service wrapper configured for zero data retention under Business Associate Agreement (BAA).